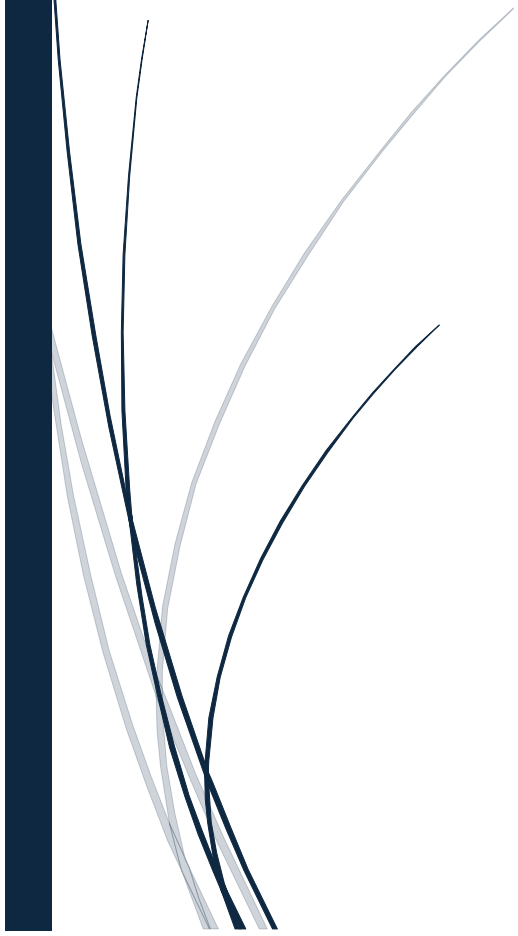
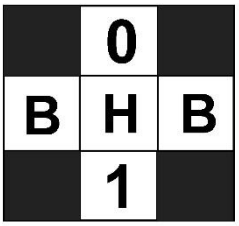
10/29/2026

AI-Enhanced vs Legacy DFIR Tools – A Comparative Efficacy Analysis

Results, Findings, and Recommendations



Whitireia and WelTec



Matt Baker, Sam Davies, Olivia Hey, Dave Sushames
BYTE HARBOR BUTCHERS

Executive Summary

This research report consolidates the findings, recommendations, and data supporting resolution to the questions: “Do LLMs belong in Digital Forensics and Incident Response (DFIR)?”, “How do DFIR reports generated by LLMs compare to those generated by a human analyst?” and “What are the challenges with augmenting DFIR analysis locally with LLMs?”. This report also includes guidance for future research in this area. LLMs are identified to provide advantages in report generation time, but their findings are often forced and unreliable, requiring validation by an expert which negates the benefits of the LLM. This report also identifies the use cases of LLMs in DFIR and the contextual requirements that cause the LLMs application to Digital Forensics (DF) and Incident Response (IR) investigations to differ in usefulness or efficacy in the real world.

Table of Contents

Executive Summary 1

Report Comparisons and Qualitative Assessment 4

Foremost 4

Nmap 5

Wireshark 5

Tools/Methodology 7

Traditional DFIR Tools 7

Case Files 7

Scripts 7

LLM Models 8

Hardware/OS 9

Design Science Research 9

Findings 10

Context Problem 10

Input Data 11

Foremost 11

Nmap 11

Wireshark 11

Generation Period 13

Corrupted Input Data 15

Thinking Enabled vs Disabled 16

Performance 19

Power 19

Temperature (Physical) 21

Memory 22

Accountability/Traceability 22

Non-comprehensive indications 22

Conclusion 24

Recommendations 25

Input Data 25

XML 25

CSV 25

Configuration 25
For Future Research 26
Data Collection / Analysis and Visualization 26
References 27
Appendices 28
Appendix A – Script Performance Data 28
Appendix B – LLM-Script Generation Time 29

Report Comparisons and Qualitative Assessment

Foremost

Feature	Definition
Tool details	Name and version of the tool.
Intro/Summary	Readable overview for a non-technical audience.
Evidence information	Name and size of the original image.
File count and types	Total number of files, and how many per type of file.
Per-file details	Name and size of each file.
Describes recovered files	What each file actually contains.
Notable findings	Clustering, large files, anything unusual or suspicious.
Limitations	Mentions shortcomings, e.g. corrupted files, fragmentation, lack of access to other tools.

	Three.dd		**Eight.dd**		**Nine.dd**	
Feature	**Human**	**LLM**	**Human**	**LLM**	**Human**	**LLM**
Tool details	Partial	Yes	Partial	Yes	Partial	Yes
Intro/Summary	Yes	Yes	Partial	Yes	Yes	Yes
Evidence information	Partial	Yes	Partial	Yes	Partial	Yes
File count and types	Yes	Yes	Yes	Yes	Yes	Yes
Per-file details	Yes	Yes	Partial	No	Partial	No
Describes recovered files	Yes	No	Yes	No	Yes	No
Notable findings	Yes	Yes	Partial	Yes	No	Yes
Limitations	Yes	Partial	Partial	No	Partial	No

LLM is very good at the tool/evidence information, while humans are not as consistent. Suggests humans don't read the information that is obvious in the audit.txt file.

Humans are much better at depth, describing the file names and contents each time, although not mentioning the file sizes every time. The LLM only mentioned the per-file details once, which is odd because it's the same script each time.

Human scores are inconsistent, likely due to difference in personal style, as each report was written by a different team member.

Nmap

Feature	Definition
Tool details	Name and version of the tool.
Intro/Summary	Readable overview for a non-technical audience.
Target information	Host/IP range scanned.
Host status	Whether each host was up.
Open ports	Port numbers, protocols, and states (open/closed/filtered)
Service details	Service names and versions on each port
OS Detection	Operating System guess for each host
Notable findings	Risky or outdated services, unusual ports, anything suspicious.
Limitations	Mentions shortcomings, e.g. filtered ports, guesses, scan scope.

	28SepServiceScan		**SOITscan**		**Nmap_example**	
Feature	**Human**	**LLM**	**Human**	**LLM**	**Human**	**LLM**
Tool details	Yes	Partial	Partial	Partial	Partial	Partial
Intro/Summary	Yes	Yes	Yes	Yes	Yes	Yes
Target information	Yes	Yes	Partial	Partial	Partial	Yes
Host status	Yes	Yes	Yes	Partial	Partial	Partial
Open ports	Yes	Partial	Partial	Partial	Partial	Partial
Service details	Yes	Partial	Partial	Partial	No	Partial
OS Detection	Partial	Partial	No	Partial	No	Partial
Notable findings	Yes	Yes	Yes	Yes	Partial	Yes
Limitations	Partial	No	No	No	Yes	No

28sepServiceScan:
The LLM-generated report presents Metasploitable host as host .22, the human says it is .21. .20/.21 have all parts closed, while human says .20 is filtered and .21 is Metasploitable. .10 is given a “moderate” risk by the LLM in findings, but “low” risk in the summary table. The reason for this is visible in Thinking Enabled vs Disabled on page 16.

Wireshark

Feature	Definition
Tool details	Name and version of the tool.
Intro/Summary	Readable overview for a non-technical audience.
Capture information	File name, capture date/duration, packet count.
Hosts/Conversations	Which IPs/hosts are communicating, and the main conversations.
Timeline	Key activity in the order it happened.
Extracted data	Credentials, files, or cleartext content recovered from the traffic.
Notable findings	Suspicious traffic, insecure protocols, anything unusual.

	Evidence01.pcap		Evidence02.pcap		Evidence03.pcap	
Feature	**Human**	**LLM**	**Human**	**LLM**	**Human**	**LLM**
Tool details	Partial	No	Partial	No	Yes	No
Intro/Summary	Yes	Yes	Yes	Yes	Yes	Yes
Capture information	Partial	Partial	Partial	Partial	Yes	Partial
Hosts/Conversations	Yes	Yes	Yes	Yes	Partial	Yes
Timeline	Partial	No	No	Partial	Yes	No
Extracted data	Yes	Yes	N/A	Partial	Yes	Partial
Notable findings	Yes	Partial	Yes	Yes	Yes	Partial

Tools/Methodology

Traditional DFIR Tools

These are the 3 tools that were selected to augment with LLMs

Tool	Purpose
Wireshark	Packet capture, packet analysis, network forensics.
Nmap	Network reconnaissance, network triage.
Foremost	File carving.

Case Files

	Foremost	Nmap	Wireshark
File Type	.DD/.TXT	.XML/.TXT	.PCAP/NG
Converted to	.TXT	.TOON	.CSV
Case File 1	Three.dd	Nmap_example.xml	evidence01.pcap
Case File 2	EIGHT.dd	SOITnetworkscan.xml	evidence02.pcapng
Case File 3	NINE.dd	28sepServiceScan.xml	evidence03.pcap

Scripts

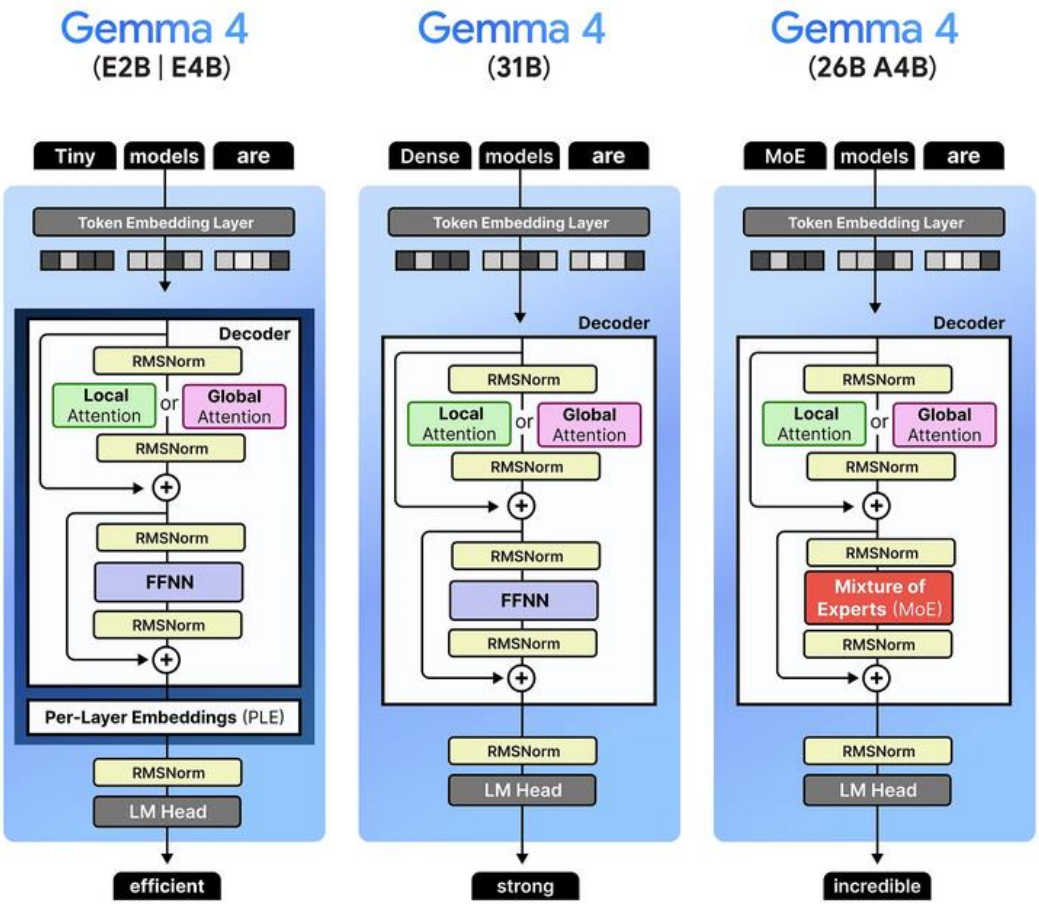
The team developed scripts to automate the tasks of processing input data, chunking the data, running sequential prompts, generating pdfs with "Pandoc", recording token usage, and deleting residual files. These scripts were developed in the python language in order to augment every traditional DFIR tool with an LLM. Other experimental models with greater theoretical capacity were created, but do not appear in the raw data found in this report. This distinction is important because some findings irrelevant to performance/metrics were gathered with these tools.

Script	Input Data Type	Context	Prompts	Thinking	Appears in Data
NinemostV1.py	Foremost	32k	1	No	Yes
SilentCartograhperV1.py	Nmap	32k	3	No	Yes
HammerheadV1.py	Wireshark	32k	3	No	Yes
SilentCartographerV3.py	Nmap	64k	5	Yes	No
HammerheadV3.py	Wireshark	64k	5	Yes	No

LLM Models

The team made two custom builds based on the Gemma4:26b model specifically for this research. Gemma4:26b is a well-regarded model for local implementation. The team tested multiple other models such as Qwen, Mistral, and Nemotron. The quickest, most lucid, least erroneous results were produced by Gemma4:26b. This can be attributed to the 26 billion parameters available to the model, interleaving attention layers, and the "Mixture of Experts" (MoE) architecture (Grootendorst, 2026), which employs selective neural subnetwork activation depending on the context to improve efficiency and accuracy (NVIDIA, n.d.).

Figure 1 - Illustration of the Decoding Modules in Respective Gemma4 Models (Grootendorst, 2026)



Gemma4 models/derivatives employed during this research.

Model	Context	Temperature	Set-seed	Parameters	Appears in Data
Gemma4:26b	*Default (4-64k)*	6	No	26 billion	No
Gemma4-df	32k	0	Yes	26 billion	Yes
Gemma4-df-64k	64k	0	Yes	26 billion	No

Hardware/OS

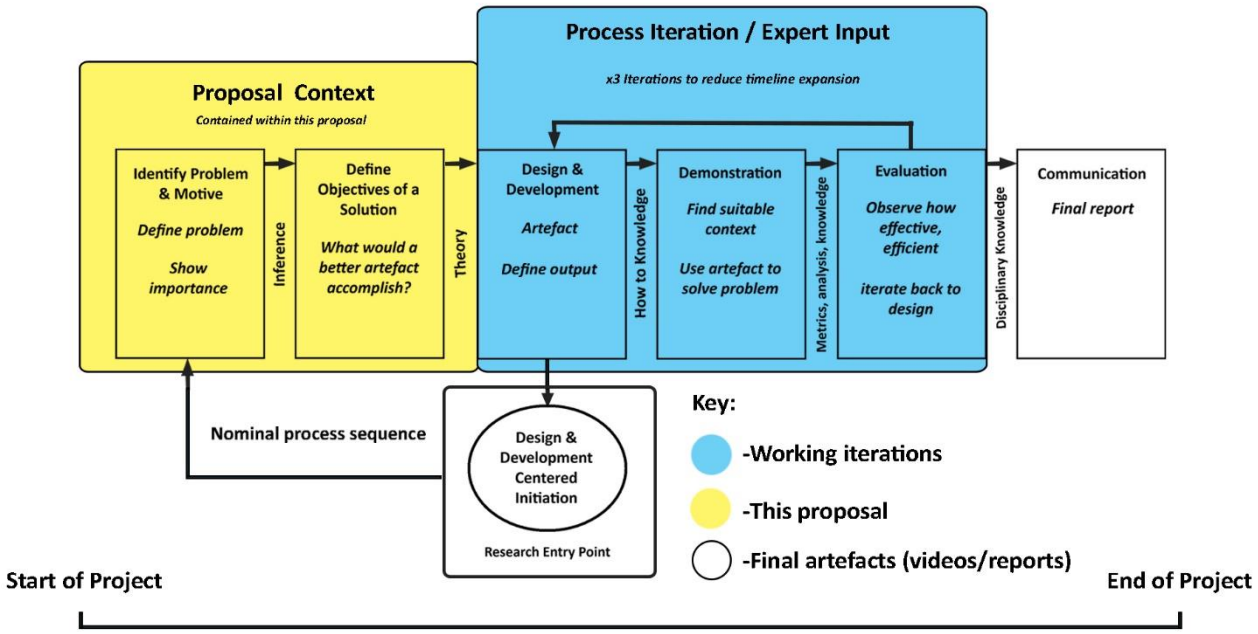
Physical components of the machine this investigation was completed with, the machine LLMs hosted and run on.

Operating System	**Kali GNU/Linux Rolling 2026.3**
CPU	13th Gen Intel(R) Core(TM) i5-13500
GPU1	Nvidia GeForce RTX 3060
GPU2	Nvidia GeForce RTX 3070
Memory (RAM)	2x 8GB DDR5

Design Science Research

The team adapted the Design Science Research (DSR) methodology which places emphasis on the continuous improvement of "artefacts" to produce products of incrementally higher quality. This style of methodology benefited the team with the value of the team's supporting technical advisor during this investigation. Artefacts such as human generated reports, LLM scripts, and data collection and presentation were continuously refined with expert input guiding the process. This methodology greatest leveraged this expert relationship to produce research and findings more in touch with real world professionals.

Figure 2 - Adapted DSR Methodology Diagram



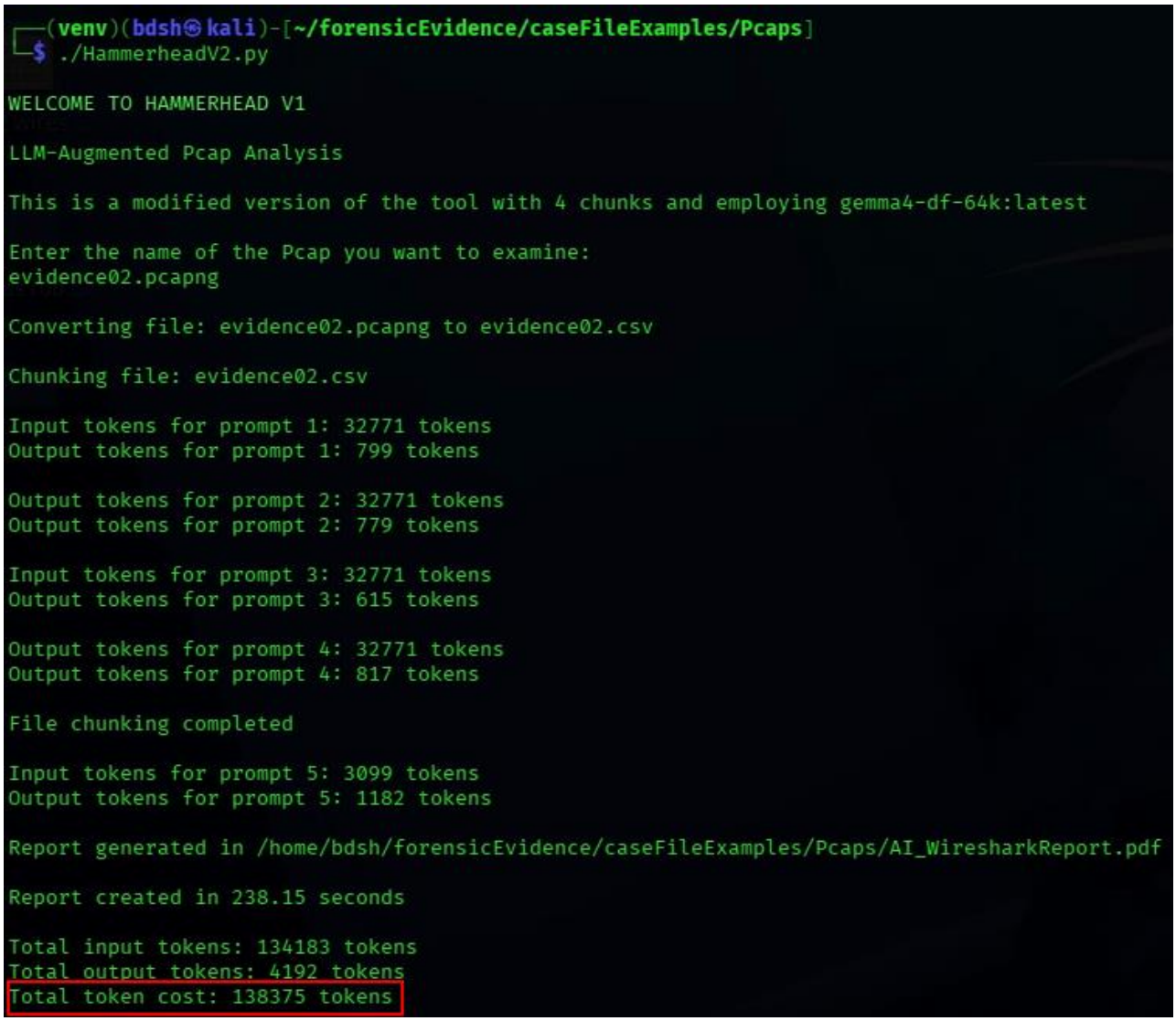
Findings

Context Problem

The most significant challenge identified in this research is unique to local deployments and reveals a problematic paradox. DFIR datasets can be massive in scale, yet the doctrine of secure analysis prescribes a necessity to command a significantly restrained available context length. This issue was addressed in this investigation by the reconfiguration and engineering of the local testing environment, model provision, and manipulation of input data. Tasks such as increasing the default Ollama context window to 32k, disabling thinking which counts towards output context, employing prompt chunking to leverage multiple prompt runs on one dataset, and reducing the token count of input data through conversion to those more consumable satisfied this end.

Experimental scripts that employ chunking of input data into 4 prompts, and the usage of an experimental 64k context model, applied to a 7442 packet pcap file, were observed to stretch out the available input context to above 134k.

Figure 3 - Experimental Script Execution Illustrating High Context Capacity



Input Data

Foremost

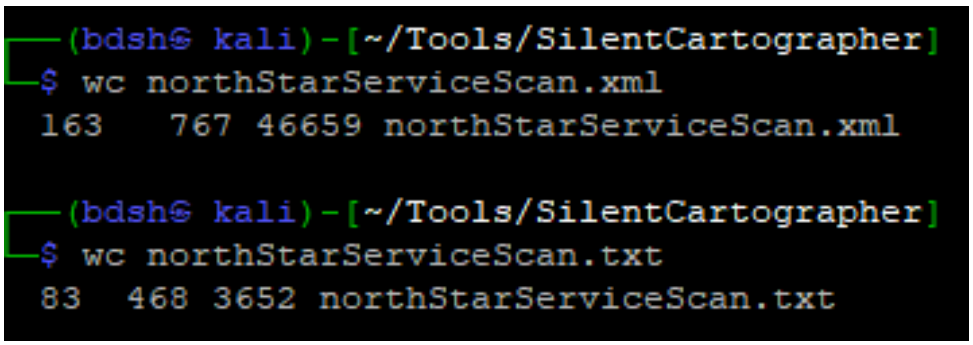
The nature of processing Foremost data with an Ollama-hosted LLM model presents some complications that are impossible to avoid. Foremost file carves produce a range of filetypes, ZIP, BMP, JPG, PNG, DOCX, PPTX, XLSX, PDF, and more, each being necessary to convert to plaintext that an LLM can read. Ollama prompts can only inspect one image at a time, so any scripted solution to analysing a foremost output directory would have to involve one prompt per image file. A human does not experience these challenges and can consume carved files as they appear in front of them.

Nmap

Nmap scans can be saved as either an XML or a raw human readable output. XML is considered a consumable format by LLMs owing to its comma separated nature, but because of the massive increase in syntactical clutter when outputting Nmap scans as XML The team observed those files to require far more context and therefore this file type becomes less ideal than the human readable output if token count is prioritized. Using .txt instead of .xml means that larger Nmap scans can be employed without requiring further context engineering.

The same Nmap service scan outputted as .xml and .txt. The xml has 163 lines to 83 for the .txt, 767 words to 468 for the .txt, and 46659 bytes to 3652 for the .txt:

Figure 4 Size Comparison of .txt and .xml Case Files



Wireshark

Wireshark presents the largest investigable datasets in the form of network packet captures. This data includes source/destination IP, frame numbers, and info column contents for every single network packet captured, in some datasets numbering in the tens of thousands. The impact of this was mitigated by exporting the Wireshark packet dissections that would be most relevant to an LLM analyst with “tshark” and feeding that data directly to an Ollama model as a CSV file. Ollama and Gemma4 still struggled greatly with the sheer size of input data despite 32k context capacity and prompt chunking, which was evident in the quality of the reports.

In this example below, the AI hallucinates the packet range it was explicitly asked for (1) and is unable to fully reassemble a message that appears from the subject typing one character at a time (2). The original message was "iknowyouarewatching" but it fails to identify this, likely due to the LLM having to keep in mind a large string of TCP packet data.

Figure 5 – LLM-Generated PCAP Report Containing Erroneous Data

Digital Forensics and Incident Response (DFIR) Report

Date of Report: Friday, October 2, 2026, 16:07:42
Subject File: `evidence03.pcap`
Analysis Scope: Packet Ranges 1168.1.10 – 891 and 862000 – 1778 (1)
Status: Closed - Benign Activity

---

1. Executive Summary

A forensic analysis of `evidence03.pcap` was conducted to identify potential indicators of compromise (IoCs), unauthorized data exfiltration, or malicious network activity. The analysis focused on two distinct packet ranges involving internal host `19_168.1.10`. The findings indicate standard, automated application behavior consistent with a user interacting with Apple-integrated media services (e.g., iTunes, Apple TV). No evidence of malicious intent, unauthorized lateral movement, or data breaches was identified.

---

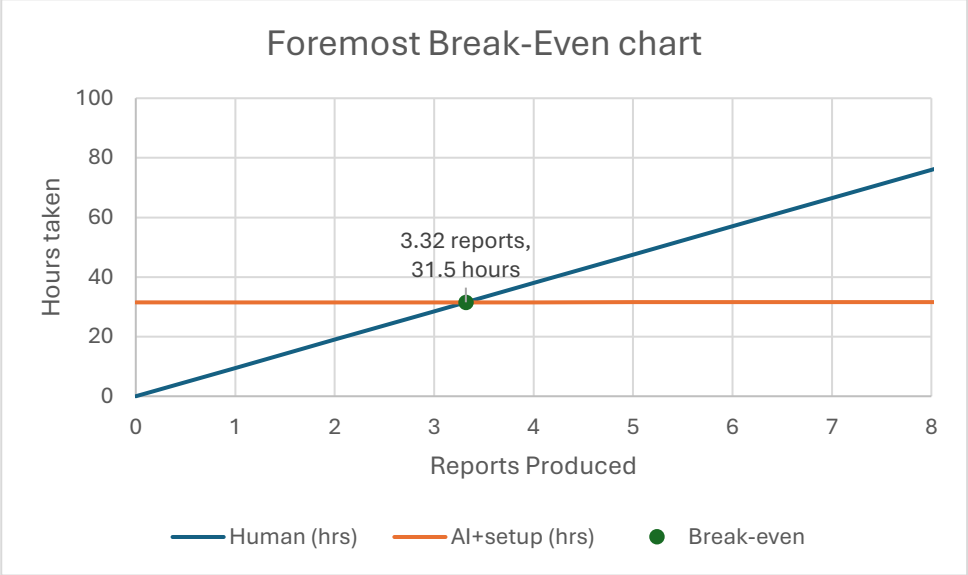
2. Incident Overview

The captured traffic is characterized by outbound-oriented HTTP/DNS activity. The primary activities observed include: * **Media Asset Retrieval:** High-volume downloading of JPEG image assets (album art/thumbnails) from Apple/Akamai Content Delivery Networks (CDNs). * **Incremental Search Functionality:** A "type-ahead" pattern where the client sends successive API requests to `ax.search.itunes.apple.com` as a search string is incrementally built (e.g., `q=iknow` → `q=iknowyou`). * **Telemetry & Analytics:** Simultaneous transmission of search strings to Appl[illegible]alytics endpoints (`66.235.132.121`) for service optimization and tracking. (2)

Generation Period

The LLMs took significantly less time to produce a report. Across all case file data (foremost output, nmap scans, packet captures) the team recorded a mean generation time of 50.75 seconds. The shortest generated report taking 21.06 seconds, the longest taking 83.52 seconds. The raw data supporting this finding can be found in Appendix A – Script Performance Data.

It is worth noting that though the execution times of the scripts is vastly shorter, these scripts took a mean time of 21.17 hours to develop. The shortest script took 13 hours, and the longest took 31.5 hours. With more sophisticated solutions development time is expected to be far greater. There is an investment of time to consider when deploying LLM automation locally. The team's assessment is that the upfront time investment reduces the benefits LLM-augmentation has over human analysts in scenarios where only 1-3 reports need to be generated depending on the type of input data.



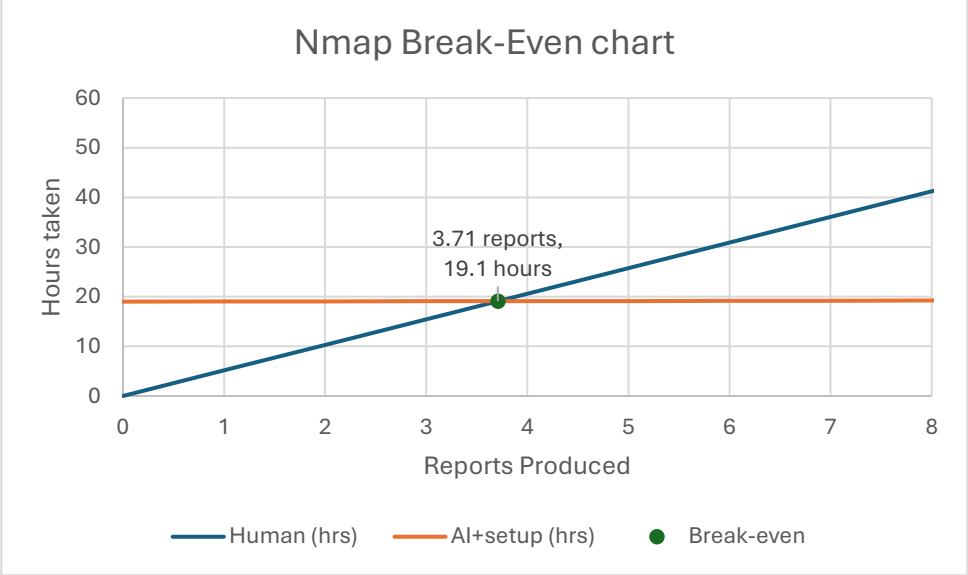
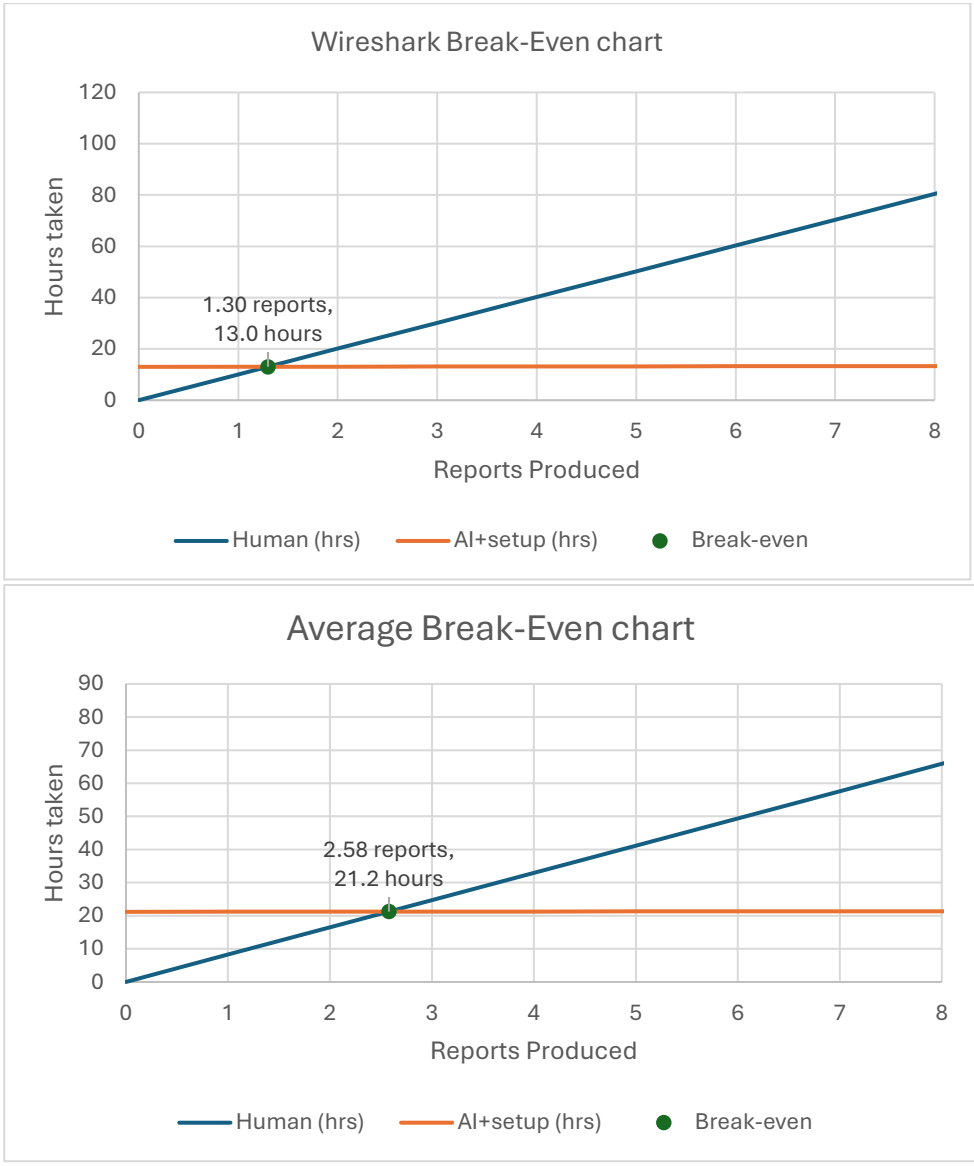


Figure 6 - Wireshark Break-Even Chart



Raw data with regards to time taken to generate LLM-augmentation scripts can be found in Appendix B – LLM-Script Generation Time on page 29.

Corrupted Input Data

Corrupted input data from fragmented disk images was investigated, in any LLM-augmented Foremost solution, images must be inspected with a vision model, the Ollama API must identify them as intact readable images first to allow the vision model to inspect them. The team found that partially corrupted images with remaining legible text will fail to pass to the LLM.

Test 1 - L0_Images.dd (no fragmentation)

The first case - this image is fully intact.

Figure 7 - 00056902.jpg, An Uncorrupted Image



Ollama will pass through and allow a vision model to examine the image

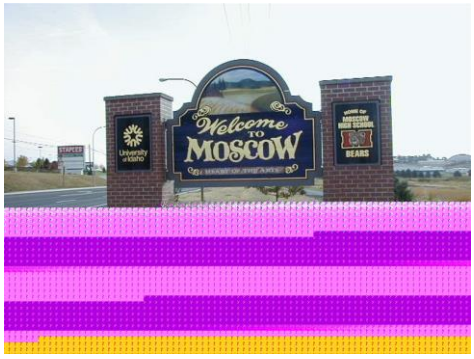
```
┌──(bdsh㉿ kali)-[~/forensicEvidence/caseFileExamples/diskImages/output/jpg]
└─$ ollama run gemma4:26b what is this image ./00056902.jpg
Added image './00056902.jpg'
Thinking...
I need to identify what the image is.

1.  **Analyze the image content:**
    *   The central focus is a large sign.
    *   The sign says "Welcome TO MOSCOW" in large, stylized letters.
    *   Below "MOSCOW", it says "HEART OF THE PANHANDLE".
    *   There's a logo on the left side of the sign: a stylized sun/flower shape and the text "University of
Idaho".
```

Test 2 – L3_Images.dd (missing fragments)

Traditional image viewing applications such as GIMP or Windows media viewer will let you view the corrupted image; a human examiner can still comprehend and extract information from the intact parts of this image.

Figure 8 - 00010000.jpg, A Partially Corrupted Image



Ollama throws an error and does not pass the image.

```
┌──(bdsh㉿ kali)-[~/forensicEvidence/caseFileExamples/diskImages/output/jpg]
└─$ ollama run gemma4:26b what is this image ./00010000.jpg
Added image './00010000.jpg'
Error: 400 Bad Request: {"error":{"code":400,"message":"Failed to load image or audio file"
,"type":"invalid_request_error"}}
```

Thinking Enabled vs Disabled

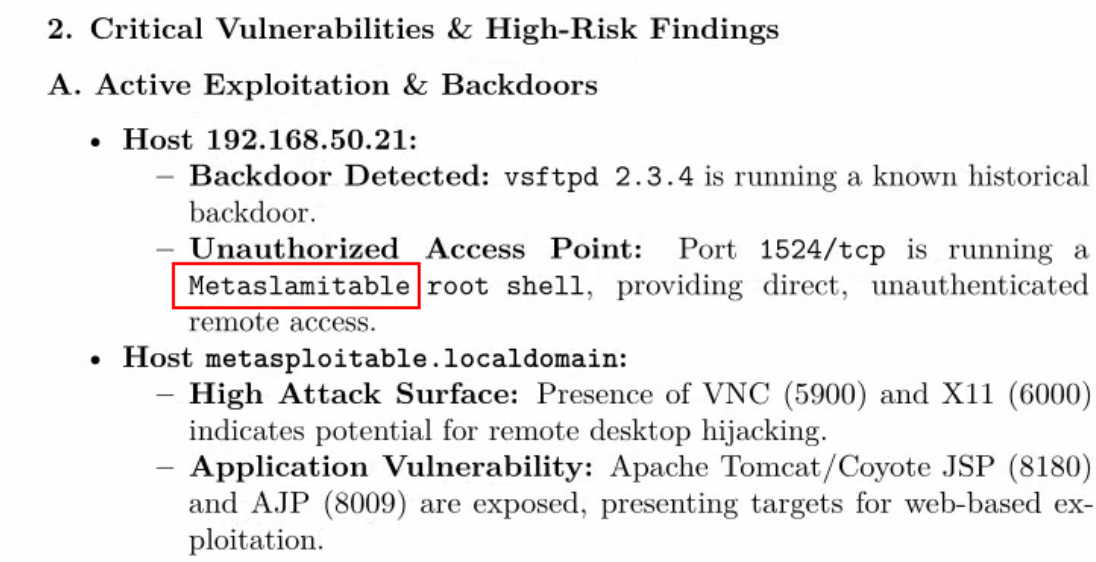
"Thinking" is a parameter that can be configured on and off with Ollama prompts. Thinking enables an Ollama hosted LLM to attempt to "reason" through the input context to reach a more accurate answer (Ollama, 2025).

The team tested for observable differences between reports generated with thinking enabled and disabled.

LLM Misspelling

Although enabled thinking cannot be assigned unequivocally as the cause for the typo, it is worth hypothesising so. This is because "thinking" forces the LLM to look over the same data multiple times, multiple reprocesses increase the risk that data is corrupted/modified by hallucination, even with the low temperature configuration of the model the team is employing.

Figure 9 - Thinking AI Making a Typo



2. Critical Vulnerabilities & High-Risk Findings

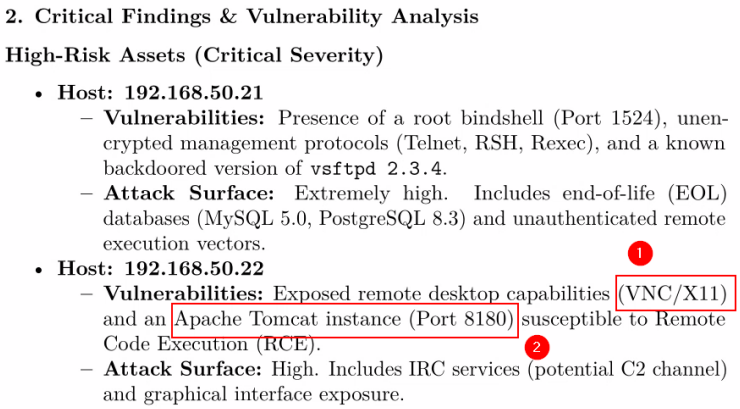
A. Active Exploitation & Backdoors

- **Host 192.168.50.21:**
  - **Backdoor Detected:** `vsftpd 2.3.4` is running a known historical backdoor.
  - **Unauthorized Access Point:** Port `1524/tcp` is running a `Metaslamitable` `root shell`, providing direct, unauthenticated remote access.
- **Host `metasploitable.localdomain`:**
  - **High Attack Surface:** Presence of VNC (5900) and X11 (6000) indicates potential for remote desktop hijacking.
  - **Application Vulnerability:** Apache Tomcat/Coyote JSP (8180) and AJP (8009) are exposed, presenting targets for web-based exploitation.

Erroneously Assigned Service Information

One extremely interesting finding is that all the reports generated with thinking disabled erroneously consider host .22 to have port 5900 (VNC service) and Apache Tomcat 8180 open (1, 2).

Figure 10 - Report Generated with Thinking Disabled

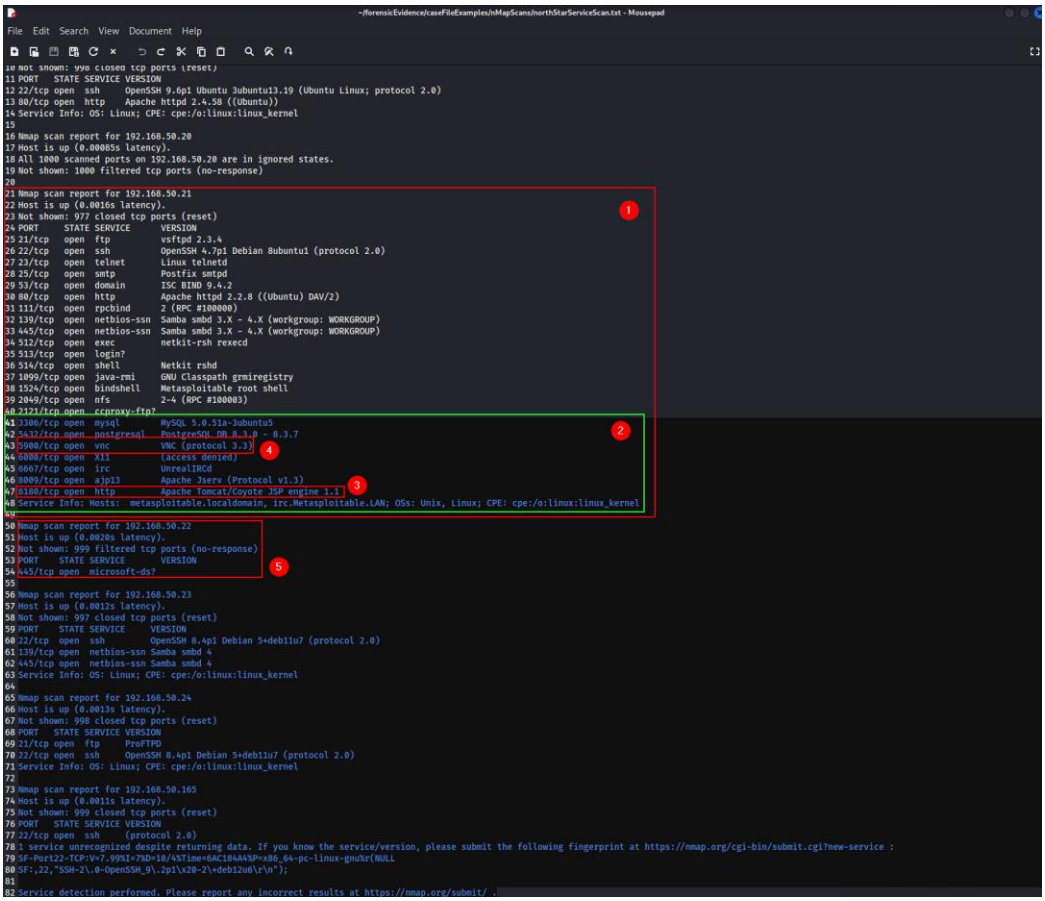


2. Critical Findings & Vulnerability Analysis

High-Risk Assets (Critical Severity)

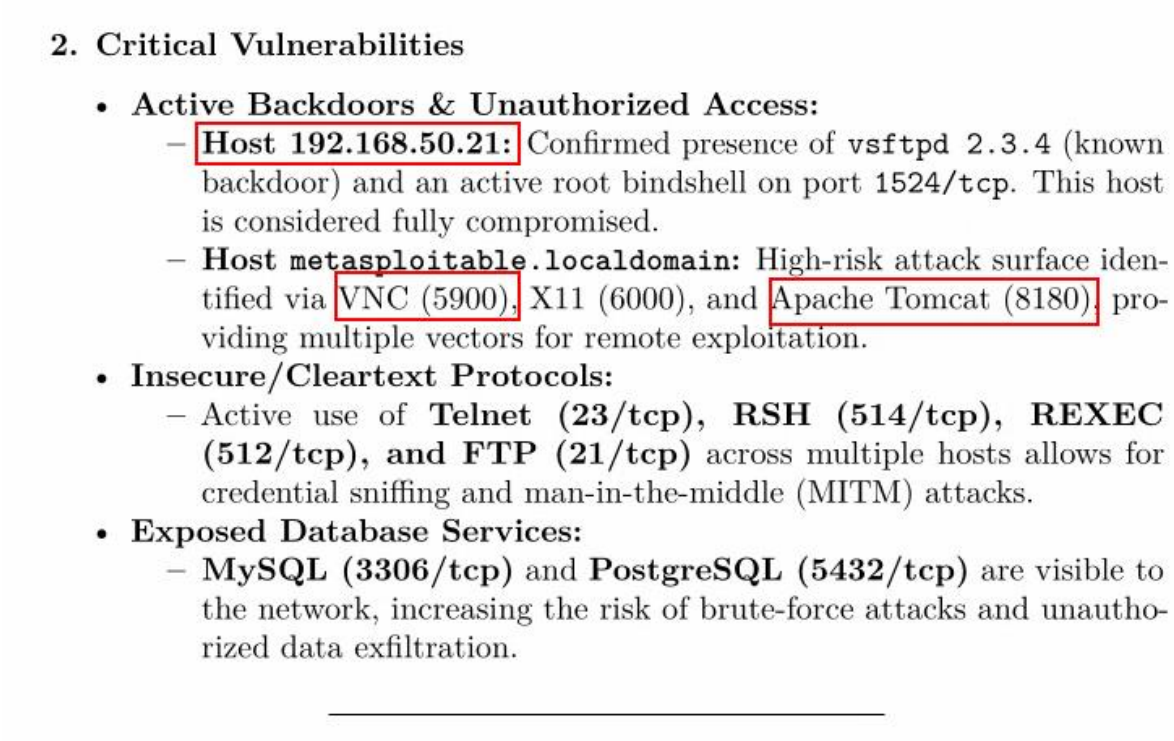
- **Host: 192.168.50.21**
  - **Vulnerabilities:** Presence of a root bindshell (Port 1524), unencrypted management protocols (Telnet, RSH, Rexec), and a known backdoored version of `vsftpd 2.3.4`.
  - **Attack Surface:** Extremely high. Includes end-of-life (EOL) databases (MySQL 5.0, PostgreSQL 8.3) and unauthenticated remote execution vectors.
- **Host: 192.168.50.22** 1
  - **Vulnerabilities:** Exposed remote desktop capabilities (VNC/X11) and an Apache Tomcat instance (Port 8180) susceptible to Remote Code Execution (RCE). 2
  - **Attack Surface:** High. Includes IRC services (potential C2 channel) and graphical interface exposure.

This is because the script prompt chunks by splitting this 82-line source scan in half (41-82 half highlighted). The scan for host .21 (metasploitable) (1) is cut off during this process (2). Because this data appears in the second chunk above host .22's scan (5), it is associated with the orphaned service info (4), (5) by the LLM. Even though the raw data from the Nmap scan points to a different objective reality.



The script with thinking enabled on its prompts however can piece together and recontextualize the chunks more accurately. This is evident in all the reports generated. They are all able to assign the exposed services to the host that the original Nmap scan identified. This finding demonstrates a limitation when working with local LLMs on DFIR datasets, and a drawback to the prompt chunking solution.

Figure 11 – LLM Report Generated with Thinking Enabled



2. Critical Vulnerabilities

- **Active Backdoors & Unauthorized Access:**
  - **Host 192.168.50.21:** Confirmed presence of `vsftpd 2.3.4` (known backdoor) and an active root bindshell on port `1524/tcp`. This host is considered fully compromised.
  - **Host `metasploitable.localdomain`:** High-risk attack surface identified via VNC (5900), X11 (6000), and Apache Tomcat (8180), providing multiple vectors for remote exploitation.
- **Insecure/Cleartext Protocols:**
  - Active use of **Telnet (23/tcp), RSH (514/tcp), REXEC (512/tcp), and FTP (21/tcp)** across multiple hosts allows for credential sniffing and man-in-the-middle (MITM) attacks.
- **Exposed Database Services:**
  - **MySQL (3306/tcp)** and **PostgreSQL (5432/tcp)** are visible to the network, increasing the risk of brute-force attacks and unauthorized data exfiltration.

Performance

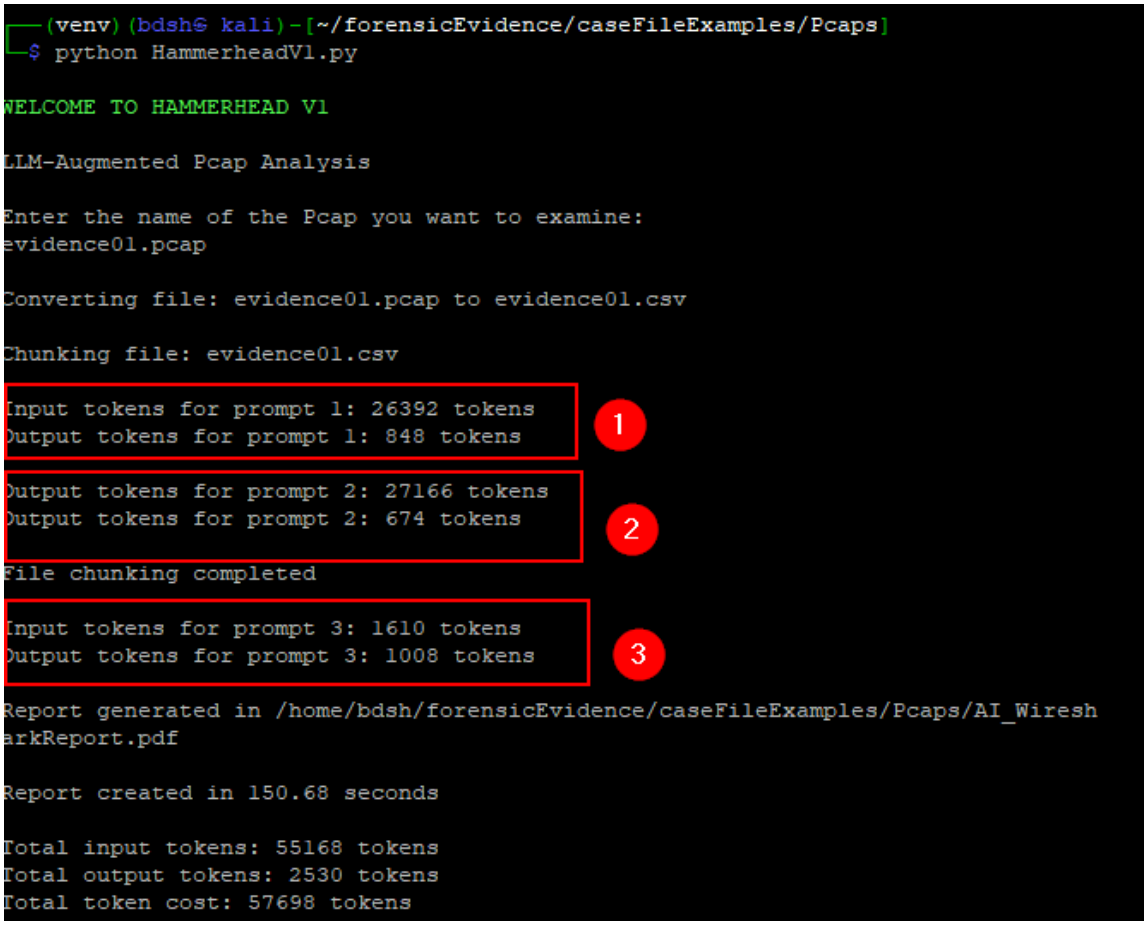
Effects on physical hardware were observed and monitored using Nvidia SMI during the execution of the team's LLM-automation scripts. Metrics assessed are GPU temperature, power draw, and Video Random Access Memory (VRAM) usage across the duration of script execution with the targets being every case file. Complete metrics can be found in Appendix A – Script Performance Data.

Power

When running LLMs locally, and especially when using multiple GPUs, it is important to have an adequately high wattage PSU to compensate for large power draws.

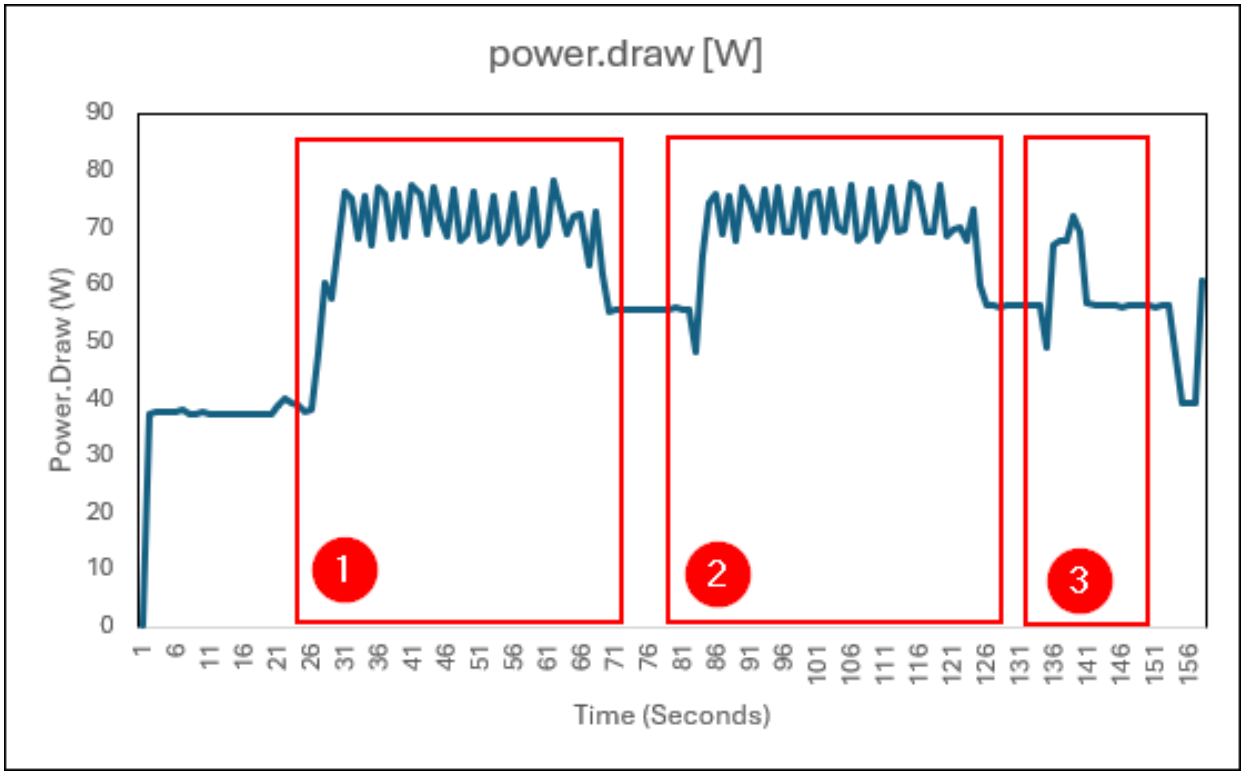
Because of the requirement that the solution "file-chunks" the input data to distribute the token requirements for each engagement, three distinct prompt calls are made to the Gemma4-df model through Ollama. The first two prompts that summarize two halves of the input data are responsible for most of the hardware demand as they are accessing large quantities of raw data. The third prompt is simply accessing the concatenated output of the first two prompts and generating the final report. Figure 12 - Respective token usage for 3 prompts shows the relatively low context requirement of the latter prompt.

Figure 12 - Respective token usage for 3 prompts



The respective drawing of resources between the three prompts correlates to sustained power draw observed on the GPUs. Figure 13- Power fluctuation on an RTX 3070 GPU during script execution clearly illustrates ~75 Watt loads sustained over a period of ~43 seconds for the two prompts with input context of ~27000 tokens, while the third prompt lasts only several seconds and draws 20% less power. With larger datasets that require more prompt chunking the expectation on a PSU would increase.

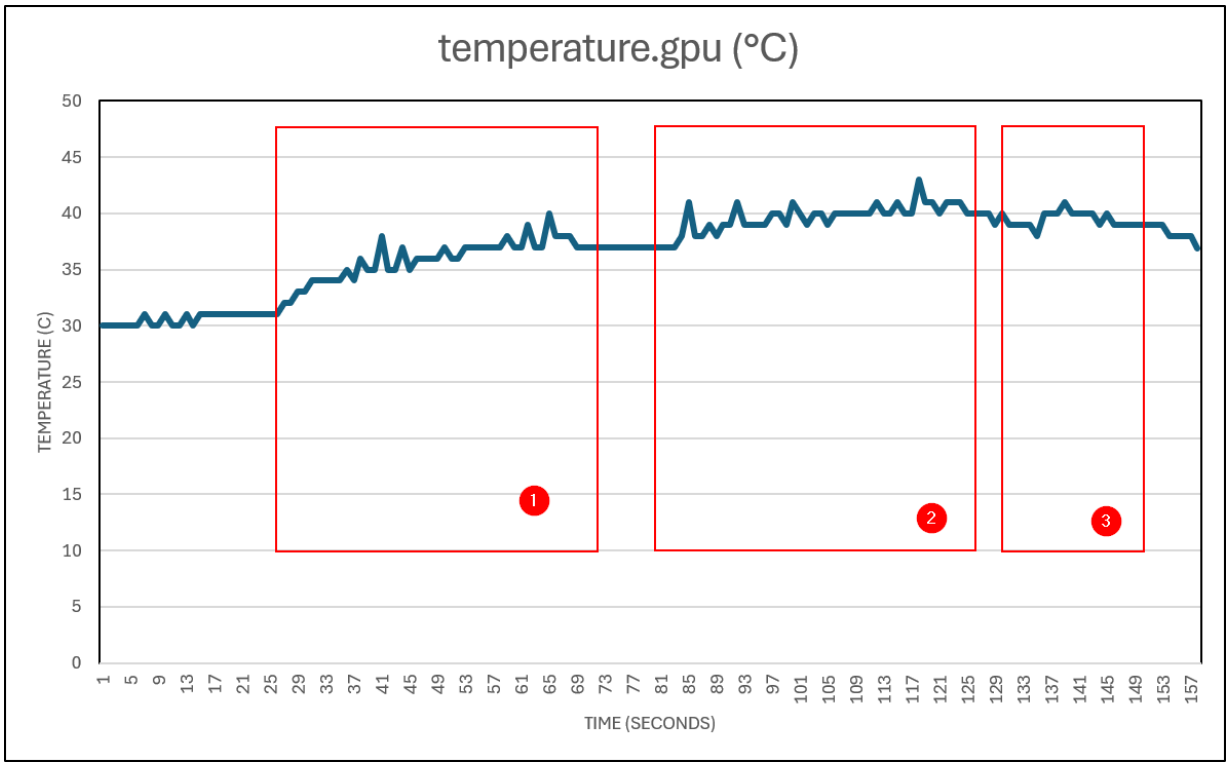
Figure 13- Power fluctuation on an RTX 3070 GPU during script execution



Temperature (Physical)

The same pattern is muted but nevertheless visible in the recorded core temperature of the GPUs under load. Because the peak temperature increases gradually as prompts are called, it is assumed that also if a larger forensic dataset is to be chunked that peak temperatures would rise even further. A 3070 will only start throttling performance at temperatures of 93 degrees. (NVIDIA, 2021) An operating temperature of 43 degrees is well within the safe operating temperature, especially for the short load durations that the team's solutions demand of LLMs.

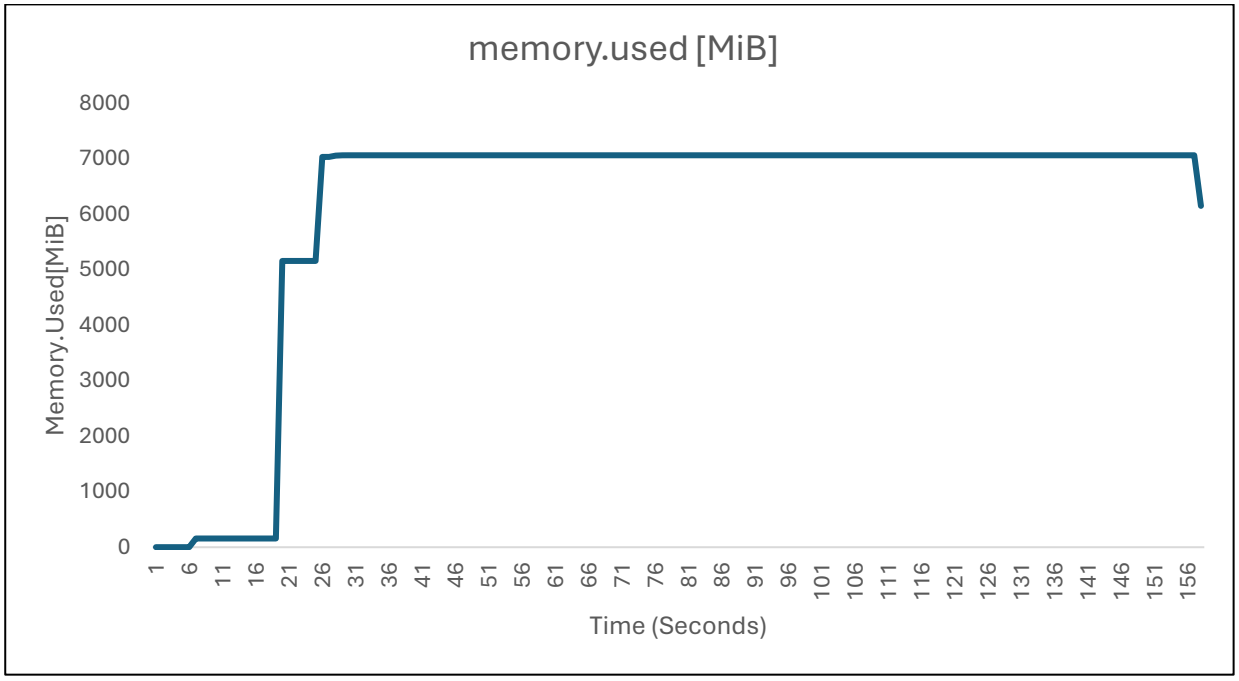
Figure 14 - Temperature fluctuation on an RTX 3070 GPU during script execution



Memory

Per-prompt memory usage remains static throughout the entire script execution.

Figure 15 - Memory usage on an RTX 3070 GPU during script execution



Accountability/Traceability

A severe epistemological challenge with employing LLMs to scrutinize and produce actionable, verifiable, and reliable findings, especially in professional settings, is that the source of their findings, and the way they came to those conclusions is obscured by the nature of their complexity. The team found that reports generated for Nmap were able to trace and track what host and what service their findings come from, which could be cross-referenced with the raw data it was asked to inspect. However often with larger datasets especially PCAPs the LLMs interest in avoiding informational redundancy would only display an incomprehensive example of a finding.

Non-comprehensive indications

The LLM reports generated were found to avoid enumerating the full truth of a situation, instead selecting a single example to represent the entire finding. This is most evident in reports that inspect pcap data. Information like this could misinform network engineers or investigators relying on the AI report as these incomplete misrepresentations undermine the scale of an attack on a network's availability. Figure 16 - Incomplete Packet Loss Data demonstrates where the LLM has failed in telling full truth by not providing a comprehensive enumeration of data such as the other network congestion that is visible in the raw data, such as those seen in Figure 17 - Duplicate ACKs and Spurious Retransmissions that Indicate Packet Loss. The human generated report demonstrated a full identification of 3 packet loss zones specifically in the transmission of packets in Evidence02.pcapng. This is identifiable in Figure 18 - Human Generated Report on Evidence02 with Mention of All Packet Loss Instances. Although this report overall is lacking the quantity of findings the LLM produced overall. No matter the input data and number of possible findings, the LLM outputs we observed will

always attempt to make a report of a seemingly predetermined length. It is also observed that LLMs findings appear “forced” to that end.

Figure 16 - Incomplete Packet Loss Data

C. Network Anomalies & Infrastructure

- **Network Instability:** Evidence of packet loss and network congestion was detected (Packets 7346–7349), specifically “TCP Previous segment not captured,” “TCP Out-of-Order,” and “TCP Dup ACK” flags during high-volume transfers.
- **Local Network Traffic:**
  - **ARP:** Host `10.0.0.194` was actively attempting to locate the default gateway (`10.0.0.1`).
  - **Discovery:** Presence of SSDP/UPnP multicast traffic (`239.255.255.250`) and IPv6 Router Advertisements.
- **External Connection:** A connection to `128.119.245.12` (Port 80) was terminated by the remote host.

Figure 17 - Duplicate ACKs and Spurious Retransmissions that Indicate Packet Loss

3380	4.355190	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=32
3381	4.355192	52.201.110.5	10.0.0.210	TCP	78 [TCP Dup ACK 3364#1] 443 → 652
3382	4.355220	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3336101
3383	4.355221	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3337561
3384	4.360312	52.201.110.5	10.0.0.210	TCP	80 [TCP Dup ACK 3364#2] 443 → 652
3385	4.360315	52.201.110.5	10.0.0.210	TCP	80 [TCP Dup ACK 3364#3] 443 → 652
3386	4.360315	52.201.110.5	10.0.0.210	TCP	68 443 → 65254 [ACK] Seq=146 Ack=
3387	4.360316	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=32
3388	4.360341	10.0.0.210	52.201.110.5	TCP	72 [TCP Spurious Retransmission]
3389	4.360341	10.0.0.210	52.201.110.5	TCP	281 [TCP Spurious Retransmission]
3390	4.360417	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3339021
3391	4.360418	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3340481
3392	4.370624	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=32
3393	4.370627	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=32
3394	4.370649	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3341941
3395	4.370650	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3343401
3396	4.370650	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3344861
3397	4.370724	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3346321
3398	4.373313	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=32
3399	4.373317	52.201.110.5	10.0.0.210	TCP	78 [TCP Dup ACK 3386#1] 443 → 652
3400	4.373349	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3347781
3401	4.373350	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3349241
3402	4.382171	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=32
3403	4.382174	52.201.110.5	10.0.0.210	TCP	80 [TCP Dup ACK 3386#2] 443 → 652
3404	4.382175	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=32

Figure 18 - Human Generated Report on Evidence02 with Mention of All Packet Loss Instances

From packets 3381 to 3403, 7202 to 7210 and 7346 to 7349, there are signs of a temporary loss of network stability as packet’s read with “TCP Dup ACK flags” and “TCP Spurious Retransmission.”

3381	4.355192	52.201.110.5	10.0.0.210	TCP	78 [TCP Dup ACK 3364#1] 443 → 65254 [AC
3382	4.355220	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3336101 Ack=1
3383	4.355221	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3337561 Ack=1
3384	4.360312	52.201.110.5	10.0.0.210	TCP	80 [TCP Dup ACK 3364#2] 443 → 65254 [AC
3385	4.360315	52.201.110.5	10.0.0.210	TCP	80 [TCP Dup ACK 3364#3] 443 → 65254 [AC
3386	4.360315	52.201.110.5	10.0.0.210	TCP	68 443 → 65254 [ACK] Seq=146 Ack=1940 W
3387	4.360316	17.132.29.71	10.0.0.210	TCP	60 443 → 65206 [ACK] Seq=1 Ack=324201 W
3388	4.360341	10.0.0.210	52.201.110.5	TCP	72 [TCP Spurious Retransmission] 65254 →
3389	4.360341	10.0.0.210	52.201.110.5	TCP	281 [TCP Spurious Retransmission] 65254
3390	4.360417	10.0.0.210	17.132.29.71	TCP	1514 65206 → 443 [ACK] Seq=3339021 Ack=1

Conclusion

In conclusion, LLM-generated DFIR report outputs can provide quick, actionable summaries and recommendations that could most realistically support a human investigation in its initial stages.

In contexts such as Incident Response (IR) that value swift triage and identification of intrusion indicators this use case is most effective. Ollama is unreliable in feeding LLMs corrupt image data from Foremost output. Lack of complete enumeration, and risks of incomplete, erroneous, or misleading data, means that in Digital Forensics (DF) contexts which value integrity, traceability, and accuracy, LLM generated outputs require an intense amount of validation by a real expert to be actionable and are thus limited in usefulness.

The risks of these limitations increase when simple prompt chunking is employed. This research found that it is realistic to provision and deploy a local LLM-enhanced DFIR solution in small business to enterprise scale environments. Running a single dedicated host with 2 mid-range GPUs can support context windows of well above 130k tokens. The initial time investment to develop proprietary tools like those demonstrated in this report presents an ultimately inconsequential challenge when met with that LLM-augmented tool's capacity to eclipse the report generation time offered by human analysts on average.

Recommendations

Input Data

XML

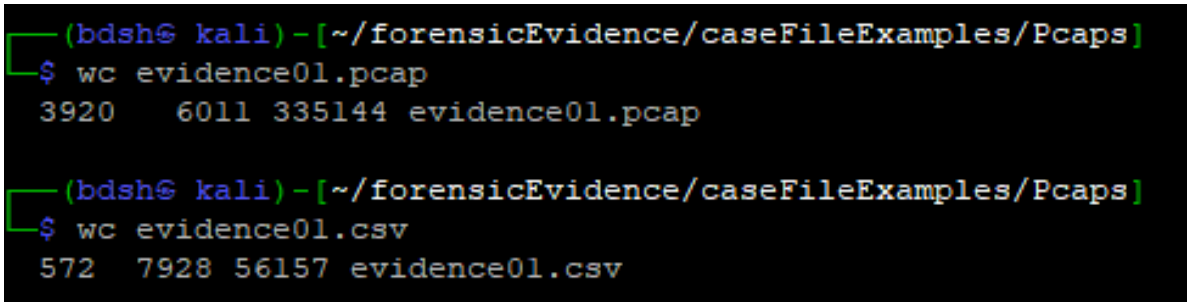
The team would recommend that XML format be avoided as an input for local LLMs to produce reports based on Nmap output. Although XML is an output format supported natively as an output by Nmap, and XML is a structure that is highly consumable by AI, .txt overall presents a lower input token count while producing reports of an identical quality.

CSV

The team recommends exporting pcap packet dissections as a CSV file, this is because LLMs find structured data formats like CSV very consumable, the size of a pcap after processing can be reduced significantly. Parsing pcaps with "tshark" also allows a great deal of configuration as you can select exactly what fields from packet dissections you want the LLM to inspect.

Case File/Format	*Lines*	*Words*	*Bytes*
evidence01.pcap	3920	6011	335144
evidence01.csv	572	7928	56157

Figure 19 - Size of Data in PCAP and CSV Formats Respectively



Configuration

If employing a similar prompt chunking solution, keep thinking on. This reduces the risk of critically erroneous findings being generated by the LLM when inspecting scattered input data.

Increase the context window override to as large as possible depending on your hardware.

Use the Gemma4:26b LLM model or a derivative. It provides the best performance and quality of output of any local model tested.

For Future Research

We hypothesise based on our research that you could theoretically do a larger scale number of prompt chunks on larger datasets. With the relatively short durations our LLMs took to produce

We recommend future researchers look at developing solutions that dynamically increase the number of chunks a script splits input data into based on the size of input data. Tweaking this mechanism would require more script engineering than our team had time for.

For Foremost-produced data especially, further investigation into corrupted content would be valuable, this presents a point where differences lie in how humans and LLMs may interpret the information they are given.

Pick a single tool to augment with LLMs. Selecting multiple tools made it difficult to define universally desirable properties of reports. It also focuses the effort of the script developer.

Investigate a hybridised testing scheme. The scripts developed in service of this research were too unsophisticated, and the context of input data was too large to produce reports that come to strong, traceable resolutions. The team would be interested in how human testers would perform and rate the usefulness of following threads generated by the limited analysis provided by LLMs.

The LLM-generated Foremost reports could be expanded to include inspection of every file. In the current script, the LLM is only reporting on the output.txt. Visual LLM models can analyse and give descriptions of images, so a solution could be developed that inspects every file/image consecutively. This was a stretch goal our team were unable to fulfil in time.

Some findings may be products of limited context windows, rather than products of the fallibility of LLMs themselves. Further research is required that takes advantage of greater context length to be sure.

Data Collection / Analysis and Visualization

Reproducing the GPU hardware performance data collected would require the use of the same command that we used which was "*nvidia-smi --query-gpu=name,memory.used,temperature.gpu,power.draw --format=csv -l 1 >> gpu_usage.csv*" this command uses Nvidia-Smi to query the GPU/s every second, collect their details and append them into a CSV file. This was run against every case file three times before the data was then converted from CSV to XLSX, cleaned, organized, averaged then graphs were generated using headers from each column (i.e. Memory. Used). If one is to revisit this investigation this is how we recorded the hardware performance metrics for AI scripting.

References

Grootendorst, M. (2026, April 4). *A Visual Guide to Gemma 4*. Retrieved from newsletter: https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-gemma-4

NVIDIA. (2021). *GeForce RTX 3070 Family*. Retrieved from Nvidia: https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3070-3070ti/

NVIDIA. (n.d.). *Mixture of Experts*. Retrieved from NVIDIA: https://www.nvidia.com/en-us/glossary/mixture-of-experts/

Ollama. (2025, May 30). *Thinking*. Retrieved from Ollama: https://ollama.com/blog/thinking

Appendices

Appendix A – Script Performance Data

GPU 1 = Nvidia GeForce RTX 3060, GPU 2 = Nvidia GeForce RTX 3070

Performance data was recorded across script execution duration 3 times per case file. Scripts used had thinking disabled, with 32k context available.

Foremost LLM Augmentation

Metrics gathered during LLM report generation for Foremost data (disk images). All figures are averages (mean). The foremost solution had no prompt chunking.

Input Data	*Time*	*Input Tokens*	*Output Tokens*	*Total Tokens*	*GPU 1 VRAM (MiB)*	*GPU 2 VRAM (MiB)*	*GPU 1 Power (W)*	*GPU 2 Power (W)*	*GPU 1 Temp (°C)*	*GPU 2 Temp (°C)*
EIGHT.dd	38.86	2020	756.67	2776.67	8448.00	7064.00	97.84	50.48	41.03	30.02
Three.dd	22.70	911	897.33	1808.33	7609	6322	87.37	48.01	37.32	30.23
NINE.dd	21.06	2020	758	2778	7362	6109.30	84.80	50.00	39.58	33.84

Nmap LLM Augmentation

Metrics gathered during LLM report generation for Nmap data (Nmap scans). All figures are averages (mean).

Input Data	*Time (avg)*	*Input Tokens*	*Output Tokens*	*Total Tokens*	*GPU 1 VRAM (MiB)*	*GPU 2 VRAM (MiB)*	*GPU 1 Power (W)*	*GPU 2 Power (W)*	*GPU 1 Temp (°C)*	*GPU 2 Temp (°C)*
Nmap_example.xml	66.62	21715.67	2726.33	24441	8062	6724.83	92.27	50.29	41.50	31.17
SOITnetworkscan.xml	48.59	17436	2254	19684	8449	7065	75.70	41.83	42.46	30.00
28sepServiceScan.xml	55.06	32149	2235.33	34384.33	8450	7066	80.69	44.99	44.94	32.23

Wireshark LLM Augmentation

This table records metrics gathered during LLM report generation for Wireshark data (packet captures). All figures are averages (mean).

Input Data	*Time (s)*	*Input Tokens*	*Output Tokens*	*Total Tokens*	*GPU 1 VRAM (MiB)*	*GPU 2 VRAM (MiB)*	*GPU 1 Power (W)*	*GPU 2 Power (W)*	*GPU 1 Temp (°C)*	*GPU 2 Temp (°C)*
evidence01.pcap	83.52	55216.33	2650.67	57867	8098.93	6759.02	93.00	51.89	45.60	33.83
evidence02.pcapng	60.94	33661.33	2498.67	36160	8448	7064	81.88	45.33	45.28	33.23
evidence03.pcap	60.40	33687.33	2476	36163.33	8449.30	7065.30	66.78	38.64	43.86	32.79

Appendix B – LLM-Script Generation Time

Script	Development Time
NinemostV1	31.5 hours
SilentCartographerV1	19 hours
HammerheadV1	13 hours
Average (mean)	21.17 hours